Informe de laboratorio 07

Tema: Programación Dinámica: Mochila 0/1 y Mochila Ilimitada

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
07	Programación Dinámica: Mochila 0/1 y Mochila Ilimitada	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	29/10/2026	02/11/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa Mochila 0/1 para un conjunto de al menos 8 objetos ingresados por el usuario e imprime tanto el valor máximo como la lista de objetos elegidos.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n, W;
    do {
        cout << "Cantidad de objetos (minimo 8): ";
        cin >> n;
    } while (n < 8);
    do {
        cout << "Capacidad de la mochila (W >= 0): ";
        cin >> W;
    } while (W < 0);
```

```
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w) {
                dp[i][w] = max(dp[i][w], valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
            }
        }
    }
    cout << "\nValor maximo (0/1): " << dp[n][W] << endl;
    vector<int> elegidos;
    int w = W;
    for (int i = n; i >= 1; i--) {
        if (dp[i][w] != dp[i - 1][w]) {
            elegidos.push_back(i);
            w -= peso[i - 1];
        }
    }
    reverse(elegidos.begin(), elegidos.end());
    cout << "Objetos elegidos:" << endl;
    int pesoTotal = 0, valorTotal = 0;
    for (size_t k = 0; k < elegidos.size(); k++) {
        int id = elegidos[k];
        cout << "  Objeto " << id << " (peso " << peso[id - 1]
             << ", valor " << valor[id - 1] << ")" << endl;
        pesoTotal += peso[id - 1];
        valorTotal += valor[id - 1];
    }
    cout << "Peso total: " << pesoTotal << " / " << W
         << "   Valor total: " << valorTotal << endl;
    return 0;
}
```

- /relleno

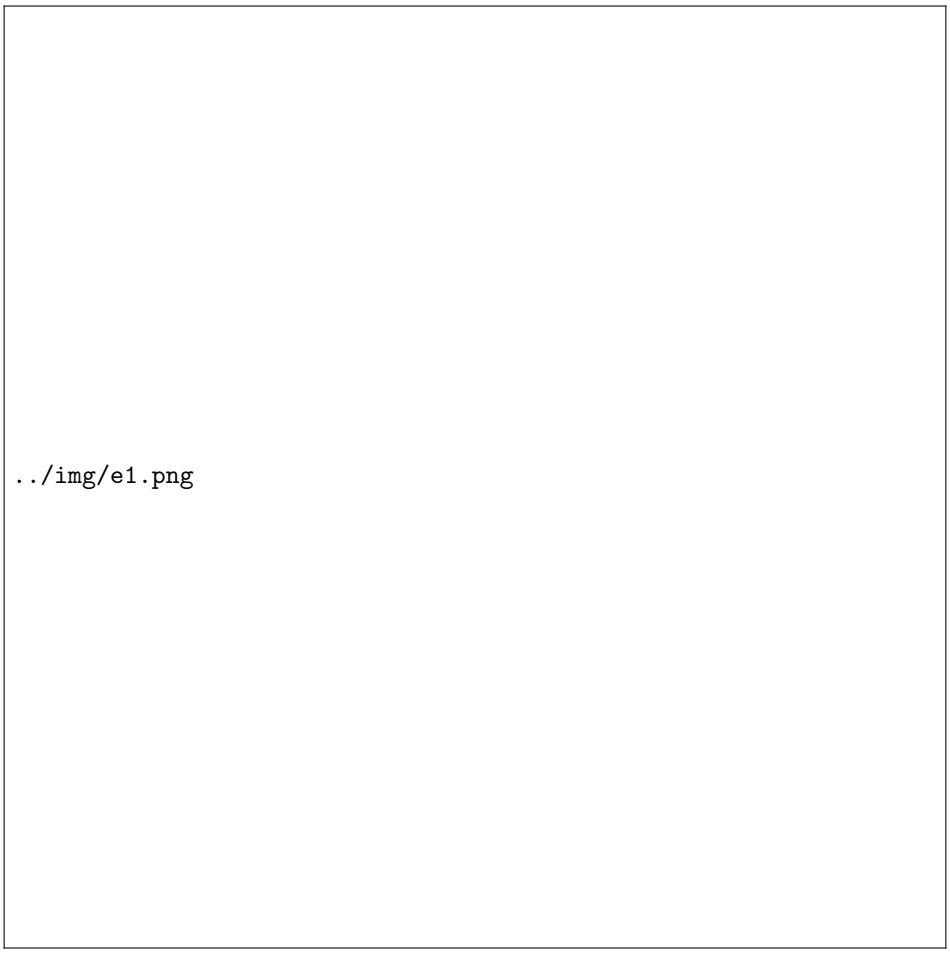


Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

Implementa Mochila Ilimitada para el clásico ≪problema del cambio de monedas≫: dado un conjunto de denominaciones y un monto objetivo, encuentra la combinación que usa la menor cantidad de monedas.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
using namespace std;
int main() {
    int n, M;
    cout << "Cantidad de denominaciones: ";
    cin >> n;
    vector<int> monedas(n);
    cout << "Ingrese las denominaciones (enteros positivos): ";
    for (int i = 0; i < n; i++) cin >> monedas[i];
    cout << "Monto objetivo: ";
    cin >> M;
```

```
    const int INF = INT_MAX / 2;
    vector<int> dp(M + 1, INF);
    dp[0] = 0;
    vector<int> eleccion(M + 1, -1);
    for (int m = 1; m <= M; m++) {
        for (int i = 0; i < n; i++) {
            int c = monedas[i];
            if (c <= m && dp[m - c] != INF) {
                if (1 + dp[m - c] < dp[m]) {
                    dp[m] = 1 + dp[m - c];
                    eleccion[m] = c;
                }
            }
        }
    }
    if (dp[M] == INF) {
        cout << "No es posible formar el monto " << M << " con esas monedas." << endl;
        return 0;
    }
    cout << "\nMinimo de monedas: " << dp[M] << endl;
    cout << "Combinacion: ";
    vector<int> usadas;
    int m = M;
    while (m > 0) {
        usadas.push_back(eleccion[m]);
        m -= eleccion[m];
    }
    sort(usadas.begin(), usadas.end(), greater<int>());
    for (size_t k = 0; k < usadas.size(); k++) {
        cout << usadas[k];
        if (k + 1 < usadas.size()) cout << " + ";
    }
    cout << " = " << M << endl;
    return 0;
}
```

- /relleno

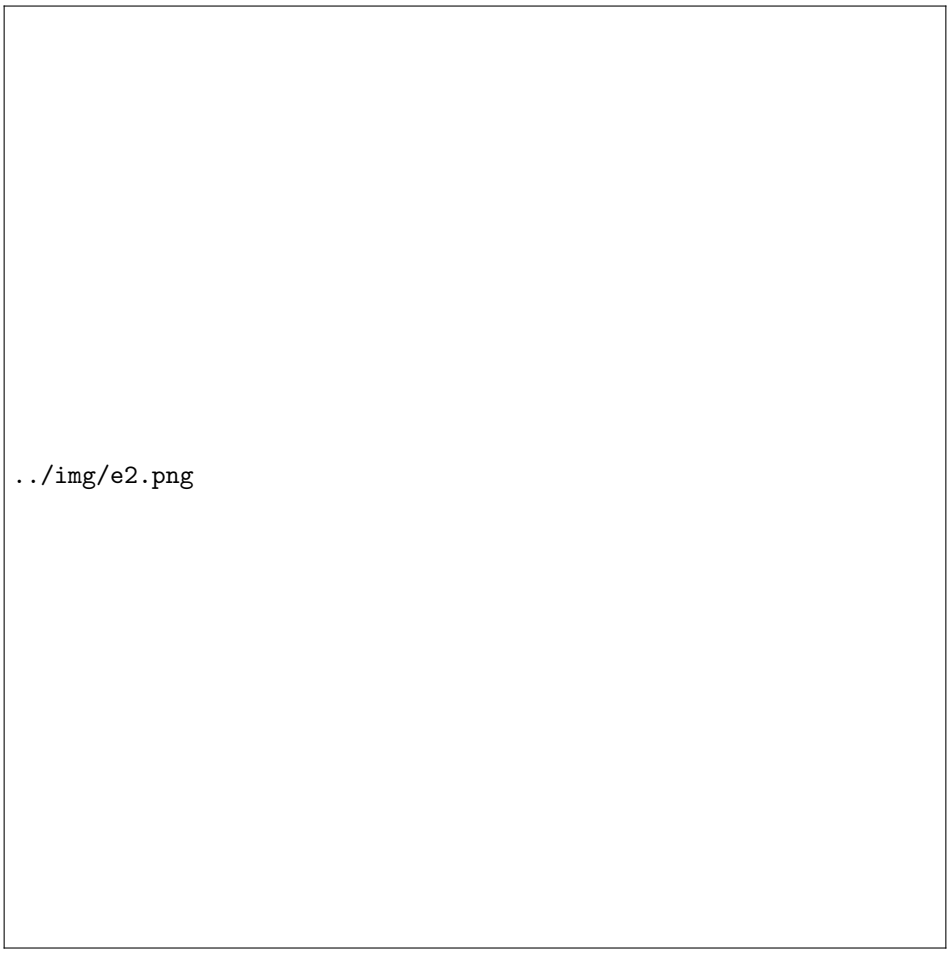


Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

Modifica la solución de Mochila 0/1 para que, además del valor máximo, muestre la tabla ‘dp‘ completa (todas las filas y columnas) y explique cómo se reconstruyó la solución.

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <algorithm>
using namespace std;
void imprimirTabla(const vector<vector<int> >& dp, int n, int W) {
    cout << "\nTabla dp[i][w]  (fila i = \"puedo usar los objetos 1..i\")\n";
    cout << "        ";
    for (int w = 0; w <= W; w++) cout << setw(4) << ("w=" + to_string(w));
    cout << "\n";
    for (int i = 0; i <= n; i++) {
        if (i == 0) cout << "i=0 (-) ";
        else        cout << "i=" << i << " obj" << i << " ";
        for (int w = 0; w <= W; w++) cout << setw(4) << dp[i][w];
        cout << "\n";
```

```
    }
}
int main() {
    int n, W;
    cout << "Cantidad de objetos: ";
    cin >> n;
    cout << "Capacidad de la mochila: ";
    cin >> W;
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w)
                dp[i][w] = max(dp[i][w],
                               valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
        }
    }
    imprimirTabla(dp, n, W);
    cout << "\nValor maximo = dp[" << n << "][" << W << "] = " << dp[n][W] << "\n";
    cout << "\nReconstruccion (se parte de la esquina dp[" << n << "][" << W << "] y se
    ↪  sube):\n";
    int w = W;
    vector<int> elegidos;
    for (int i = n; i >= 1; i--) {
        cout << "  dp[" << i << "][" << w << "] = " << dp[i][w]
             << "  vs  dp[" << (i - 1) << "][" << w << "] = " << dp[i - 1][w];
        if (dp[i][w] != dp[i - 1][w]) {
            cout << "  -> son DISTINTOS: se llevo el objeto " << i
                 << " (peso " << peso[i - 1] << "), capacidad pasa de "
                 << w << " a " << (w - peso[i - 1]) << "\n";
            elegidos.push_back(i);
            w -= peso[i - 1];
        } else {
            cout << "  -> son IGUALES: NO se llevo el objeto " << i << "\n";
        }
    }
    reverse(elegidos.begin(), elegidos.end());
    cout << "\nObjetos elegidos: ";
    if (elegidos.empty()) cout << "(ninguno)";
    for (size_t k = 0; k < elegidos.size(); k++) cout << elegidos[k] << " ";
    cout << endl;
    return 0;
}
```

- /relleno

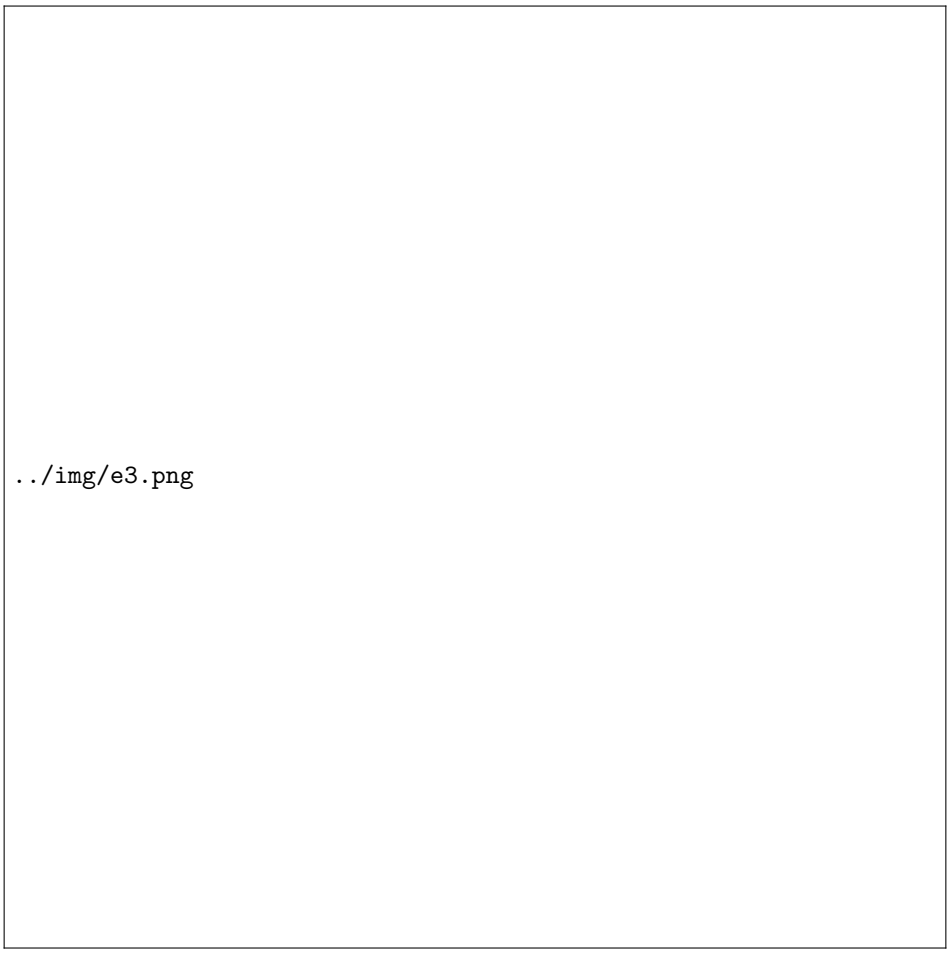


Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

Compara el uso de memoria de Mochila 0/1 (tabla 2D) contra una versión optimizada que use solo un arreglo 1D recorrido de derecha a izquierda. Explica por qué ese recorrido evita reutilizar el mismo objeto más de una vez.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int mochila2D(const vector<int>& peso, const vector<int>& valor, int W) {
    int n = peso.size();
    vector<vector<int> > dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (peso[i - 1] <= w)
                dp[i][w] = max(dp[i][w], valor[i - 1] + dp[i - 1][w - peso[i - 1]]);
        }
    return dp[n][W];
```

```cpp
}
int mochila1D(const vector<int>& peso, const vector<int>& valor, int W) {
    int n = peso.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        for (int w = W; w >= peso[i]; w--)
            dp[w] = max(dp[w], valor[i] + dp[w - peso[i]]);
    }
    return dp[W];
}
int mochila1D_alReves(const vector<int>& peso, const vector<int>& valor, int W) {
    int n = peso.size();
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < n; i++)
        for (int w = peso[i]; w <= W; w++)
            dp[w] = max(dp[w], valor[i] + dp[w - peso[i]]);
    return dp[W];
}
int main() {
    int n, W;
    cout << "Cantidad de objetos: ";
    cin >> n;
    cout << "Capacidad de la mochila: ";
    cin >> W;
    vector<int> peso(n), valor(n);
    for (int i = 0; i < n; i++) {
        cout << "Objeto " << (i + 1) << " -> peso y valor: ";
        cin >> peso[i] >> valor[i];
    }
    int r2 = mochila2D(peso, valor, W);
    int r1 = mochila1D(peso, valor, W);
    int rMal = mochila1D_alReves(peso, valor, W);
    cout << "\nValor maximo con tabla 2D          : " << r2 << endl;
    cout << "Valor maximo con arreglo 1D (der->izq): " << r1 << endl;
    cout << "Valor con arreglo 1D (izq->der, MAL)   : " << rMal
         << (rMal != r1 ? "   <- distinto: reutiliza objetos" : "   (coincide en este
         ↪  caso)") << endl;
    cout << (r1 == r2 ? "Las versiones correctas coinciden.\n" : "ERROR: no coinciden.\n");
    long long mem2D = (long long)(n + 1) * (W + 1) * sizeof(int);
    long long mem1D = (long long)(W + 1) * sizeof(int);
    cout << "\nMemoria tabla 2D: (n+1)*(W+1) = " << (n + 1) << "*" << (W + 1)
         << " enteros = " << mem2D << " bytes" << endl;
    cout << "Memoria arreglo 1D: (W+1) = " << (W + 1)
         << " enteros = " << mem1D << " bytes" << endl;
    cout << "Ahorro: " << (n + 1) << " veces menos memoria (O(n*W) -> O(W))." << endl;
    cout << "Nota: con 1D se pierde la tabla completa, por lo que ya no se puede\n"
            "reconstruir facilmente que objetos se eligieron.\n";
    return 0;
}
```

- /relleno

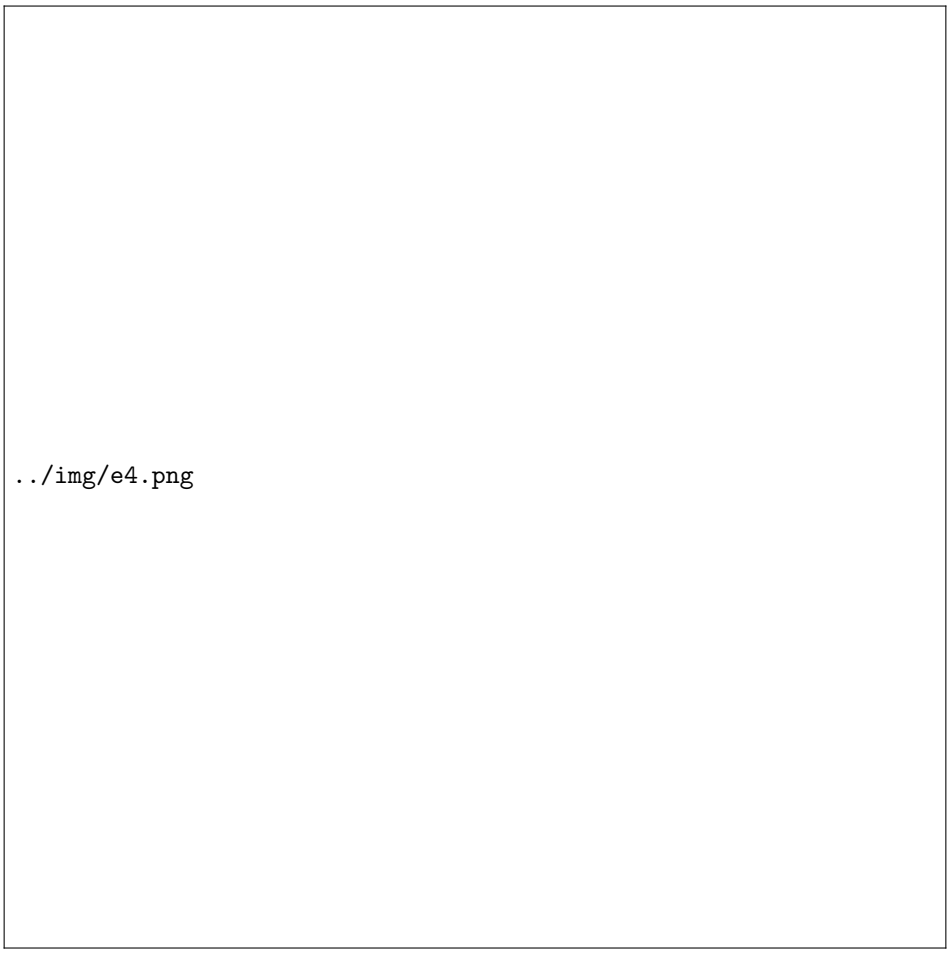


Figura 4: Ejecución del ejercicio 4

1.5. Ejercicio 5

Aplica Mochila 0/1 a un caso real: dado un presupuesto fijo y una lista de posibles compras (con costo y beneficio estimado), determina la combinación que maximiza el beneficio sin exceder el presupuesto.

```cpp
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;
int main() {
    int n, presupuesto;
    cout << "Presupuesto disponible (entero, ej. soles): ";
    cin >> presupuesto;
    cout << "Cantidad de compras posibles: ";
    cin >> n;
    vector<string> nombre(n);
    vector<int> costo(n), beneficio(n);
```

```
    cin.ignore();
    for (int i = 0; i < n; i++) {
        cout << "Compra " << (i + 1) << " - nombre: ";
        getline(cin, nombre[i]);
        cout << "           costo y beneficio estimado (ej. 1200 8): ";
        cin >> costo[i] >> beneficio[i];
        cin.ignore();
    }
    vector<vector<int> > dp(n + 1, vector<int>(presupuesto + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int p = 0; p <= presupuesto; p++) {
            dp[i][p] = dp[i - 1][p];
            if (costo[i - 1] <= p)
                dp[i][p] = max(dp[i][p],
                               beneficio[i - 1] + dp[i - 1][p - costo[i - 1]]);
        }
    }
    vector<int> elegidas;
    int p = presupuesto;
    for (int i = n; i >= 1; i--) {
        if (dp[i][p] != dp[i - 1][p]) {
            elegidas.push_back(i - 1);
            p -= costo[i - 1];
        }
    }
    reverse(elegidas.begin(), elegidas.end());
    cout << "\n=== Plan de compras optimo ===\n";
    int gasto = 0;
    for (size_t k = 0; k < elegidas.size(); k++) {
        int j = elegidas[k];
        cout << " - " << left << setw(22) << nombre[j]
             << " costo " << setw(6) << costo[j]
             << " beneficio " << beneficio[j] << "\n";
        gasto += costo[j];
    }
    if (elegidas.empty()) cout << " (ninguna compra cabe en el presupuesto)\n";
    cout << "Gasto total: " << gasto << " de " << presupuesto
         << "   Sobrante: " << (presupuesto - gasto) << "\n";
    cout << "Beneficio total maximo: " << dp[n][presupuesto] << endl;
    return 0;
}
```

- /relleno

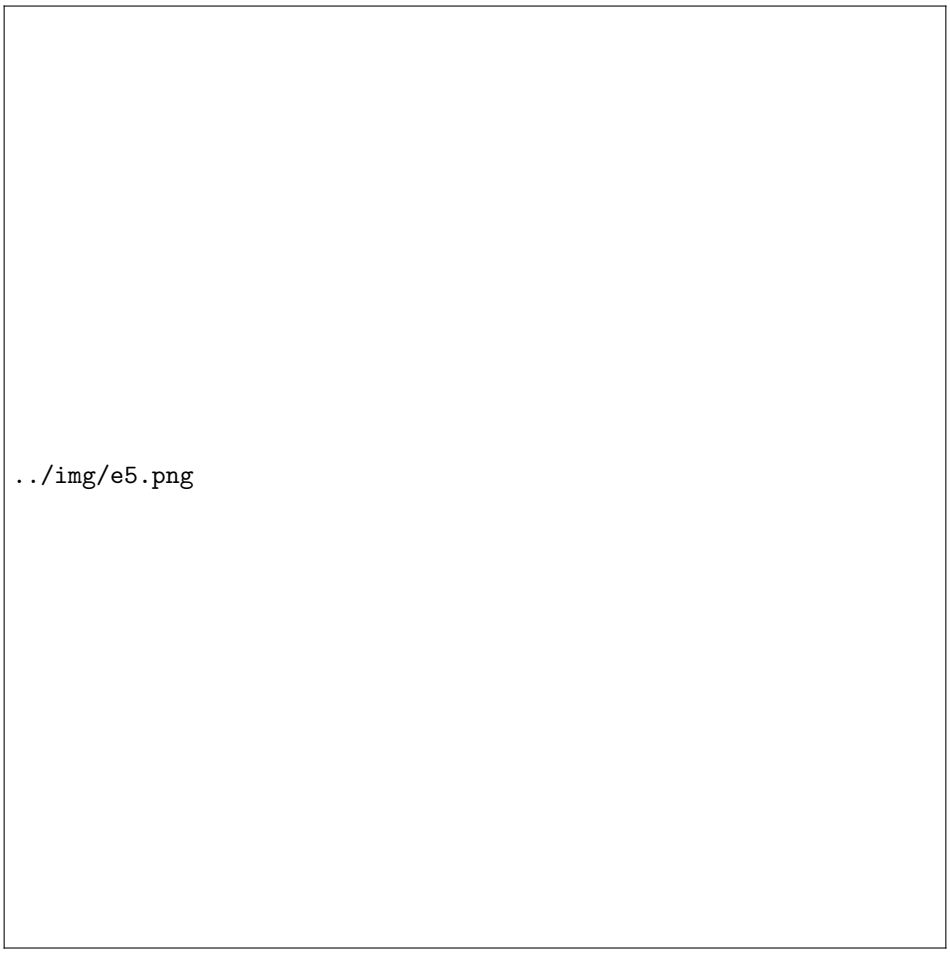


Figura 5: Ejecución del ejercicio 5

2. Cuestionario

1. ¿Por qué Mochila 0/1 necesita una tabla bidimensional, mientras que Mochila Ilimitada se resuelve con un arreglo unidimensional?
2. En Mochila 0/1, ¿qué representa exactamente el valor `dp[i][w]`?
3. ¿Por qué en el ejemplo resuelto Mochila Ilimitada obtiene un valor mayor que Mochila 0/1 con la misma capacidad?
4. ¿Cómo se podría adaptar Mochila 0/1 para permitir llevar cada objeto hasta un número limitado de veces (por ejemplo, máximo 3 unidades de cada uno)?
5. ¿Qué papel cumple el paso B (casos base) de SRTBOT en ambas variantes de la mochila?

3. Conclusiones